

《面向对象程序设计实践》

课程论文

题目：	通讯录管理程序
专业：	软件工程
班级：	2024 级 5 班
小组成员 1：	202415110210 黄奕恒
2：	202425220514 李春良
3：	
指导老师：	彭红星
提交时间：	2024 年 5 月 17 日

目录

一、系统分析.....	4
1.1 问题描述.....	4
1.2 系统功能分析.....	4
1.2.1 联系人管理功能.....	4
1.2.2 分组管理功能.....	4
1.2.3 搜索功能.....	4
1.2.4 数据持久化功能.....	5
1.2.5 导入导出功能.....	5
1.2.6 界面交互功能.....	5
1.3 开发平台及工具介绍.....	5
二、系统设计.....	6
2.1 系统总体结构设计.....	6
2.1.1 系统总体架构.....	6
2.1.2 系统功能模块设计.....	7
2.1.3 系统总体流程设计.....	7
2.2 系统各个类及类之间的关系设计.....	8
2.2.1 Contact 联系人实体类.....	8
2.2.2 ContactRepository 联系人数据仓库类.....	8
2.2.3 GroupRepository 分组数据仓库类.....	9
2.2.4 SearchService 搜索筛选服务类.....	9
2.2.5 ContactService 联系人业务服务类.....	9
2.2.6 GroupService 分组业务服务类.....	9
2.2.7 CsvUtil CSV 文件工具类.....	9
2.2.8 ErrorHandler 全局异常处理类.....	10
2.2.9 Constants 全局常量工具类.....	10
2.2.10 AppController 全局总控制器.....	10
2.2.11 ContactController 联系人控制器.....	10
2.2.12 GroupController 分组控制器.....	10
2.2.13 MainFrame 系统主窗口界面类.....	11
2.2.14 GroupTreePanel 分组树面板类.....	11
2.2.15 ContactTablePanel 联系人表格面板类.....	11
2.2.16 ContactDialog 联系人编辑弹窗类.....	11
2.2.17 类之间的关系图.....	11
2.3 数据存储的设计.....	12
2.3.1 整体存储方案设计.....	12
2.3.2 持久化文件规划与作用.....	12
2.3.3 内存数据结构设计.....	12
2.3.4 序列化存储机制设计.....	12
2.3.5 CSV 导入导出存储格式设计.....	13
2.4 界面设计.....	13
2.4.1 整体界面布局.....	13
2.4.2 主界面设计.....	14

2.4.3 编辑弹窗设计.....	14
2.4.4 交互与视觉设计规范.....	14
三、系统实现.....	14
3.1 核心功能实现.....	14
3.1.1 联系人实体类实现.....	14
3.1.2 数据持久化功能实现.....	15
3.1.3 联系人新增功能实现.....	15
3.1.4 联系人修改功能实现.....	16
3.1.5 联系人删除及批量删除实现.....	16
3.1.6 分组管理功能实现.....	16
3.1.7 模糊搜索与分组筛选实现.....	17
3.1.8 CSV 导入导出功能实现.....	17
3.1.9 全局异常与弹窗提示实现.....	18
四、系统测试.....	18
4.1 模块测试.....	18
4.1.1 联系人管理模块测试.....	18
4.1.2 分组管理模块测试.....	19
4.1.3 搜索与筛选模块测试.....	19
4.1.4 CSV 导入导出模块测试.....	20
4.1.5 数据持久化模块测试.....	20
4.2 系统测试.....	20
4.2.1 测试用例：系统完整功能流程测试.....	20
4.2.2 测试用例：异常场景测试.....	21
五、系统运行界面.....	21
5.1 主界面.....	21
5.2 新增/修改联系人.....	22
5.3 分组管理.....	22
5.4 搜索结果.....	25
5.5 导入导出.....	26
5.6 批量操作.....	28
六、总结.....	28

一、系统分析

1.1 问题描述

在日常学习、工作与生活中，个人需要管理大量联系人信息，包括姓名、电话、邮箱、单位、分组、备注等。传统手动记录方式效率低、易丢失、难以快速检索与分类管理。为解决上述问题，开发一款桌面端通讯录管理系统，实现联系人的新增、修改、删除、分组、搜索、导入导出与持久化存储，提升信息管理效率与安全性。

1.2 系统功能分析

1.2.1 联系人管理功能

联系人管理是本系统的核心模块，主要用于实现联系人信息的新增、编辑、删除以及批量操作等功能。用户可以通过图形界面快速录入联系人姓名、电话号码、邮箱地址、所属分组等信息，并将数据保存至系统中。

在新增联系人时，系统会对输入内容进行基本合法性校验，例如电话号码格式、邮箱格式以及姓名是否为空等，从而提高数据的规范性与准确性。对于已有联系人，用户可以随时进行修改操作，更新联系方式或补充其他信息。

系统同时支持单个联系人删除与批量删除功能。用户可通过复选框选中多个联系人，一次性完成删除操作，从而提高管理效率。在删除前，系统会弹出确认提示框，避免误操作导致数据丢失。

此外，联系人信息以表格形式展示，用户能够更加直观地查看所有联系人数据，提高系统的可读性与操作便捷性。

1.2.2 分组管理功能

为了方便用户对联系人进行分类管理，系统提供了完善的分组管理功能。用户可以根据实际需求创建不同的联系人分组，例如“家人”“同学”“同事”“客户”等。

系统支持新增分组与删除分组操作。当用户创建新分组后，可将联系人添加到指定分组中，实现联系人分类管理；同时，也支持将联系人从某一分组中移出，便于动态调整联系人归属关系。

在界面设计方面，系统采用树形结构展示分组信息，用户可以通过点击不同分组快速查看对应联系人列表。通过分组筛选功能，系统能够仅显示当前分组中的联系人数据，从而减少信息干扰，提高查找效率。

该功能有效解决了联系人数量较多时难以管理的问题，使联系人信息更加清晰、有序。

1.2.3 搜索功能

随着联系人数量不断增加，快速定位目标联系人变得尤为重要。因此，本系统设计了关键词模糊搜索功能。

用户可以通过输入联系人姓名、电话号码或邮箱地址中的任意关键字进行查询，系统会自动匹配符合条件的联系人信息，并实时显示搜索结果。该搜索方式无需输入完整内容，提高了搜索灵活性与使用效率。

例如，用户输入姓名中的部分字符、电话号码中的部分数字或邮箱前缀，系统即可筛选出相关联系人。搜索结果会动态更新，帮助用户快速定位目标信息。

模糊搜索功能不仅提高了系统的实用性，也增强了用户的交互体验，使联系人查询更加快捷、智能。

1.2.4 数据持久化功能

为了保证联系人数据不会因程序关闭而丢失，系统实现了数据持久化功能。系统采用本地文件存储机制，将联系人信息保存至指定文件中。

当用户新增、修改或删除联系人后，系统会自动更新数据文件，实现数据实时保存。在程序重新启动时，系统会自动读取本地存储的数据，并恢复联系人列表与分组信息，无需用户重新录入。

该功能有效保障了数据安全性与系统稳定性，提高了软件的实用价值。同时，也为后续数据扩展与系统升级提供了基础支持。

1.2.5 导入导出功能

为了方便数据迁移、备份以及跨平台使用，系统提供 CSV 格式的数据导入与导出功能。

在导入功能中，用户可以选择本地 CSV 文件，系统会自动解析文件内容，并批量生成联系人信息，从而避免手动逐条录入，提高数据录入效率。

在导出功能中，系统支持将当前联系人数据保存为 CSV 文件，用户可将文件用于备份、打印或在其他系统中继续使用。

CSV 文件格式具有通用性强、兼容性高的特点，因此该功能增强了系统的数据共享能力与扩展能力。

1.2.6 界面交互功能

为了提升用户操作体验，系统采用可视化图形界面设计，并提供多种友好的交互功能。系统使用表格组件展示联系人数据，使联系人信息更加直观清晰；采用树形结构显示分组信息，便于用户快速切换联系人分类。

在批量操作方面，系统提供复选框全选功能，用户可一键选中所有联系人，实现批量删除等操作，减少重复点击。

此外，系统还设计了多种提示弹窗，例如新增成功提示、删除确认提示、错误输入警告等，使用户能够及时获取操作反馈，降低误操作风险。

整体界面布局简洁明了，操作逻辑清晰，能够满足普通用户对联系人管理系统的日常使用需求。

1.3 开发平台及工具介绍

本次通讯录管理程序全程采用纯 Java 技术栈开发，无第三方复杂框架依赖，贴合面向对象课程学习实践目标，具体开发环境与工具如下：

核心开发语言：Java，该语言面向对象编程特性成熟完善，具备极强的跨平台运行能力，语法严谨规范，适合桌面应用程序开发。

集成开发环境：IntelliJ IDEA，主流 Java 开发工具，自带智能代码提示、语法错误检测、断点调试、项目结构管理等实用功能，能够大幅提升代码编写与程序调试效率。

图形界面开发框架：Java Swing，Java 原生桌面 GUI 开发框架，无需额外配置依赖库，可快速搭建窗口、按钮、表格、树形菜单、弹窗表单等各类桌面界面组件。

本地数据存储方式：采用 Java 对象序列化技术，生成.data 格式本地文件，直接将程序内实体对象完整保存至本地，简化数据读写逻辑。

数据交互格式：CSV 通用表格文件，格式简单易编辑，适配各类办公软件，满足通讯录数据导入导出需求。

项目构建运行方式：纯 Java 源代码编译运行，依托 JDK 运行环境即可启动程序，部署简单，适配 Windows 主流操作系统。

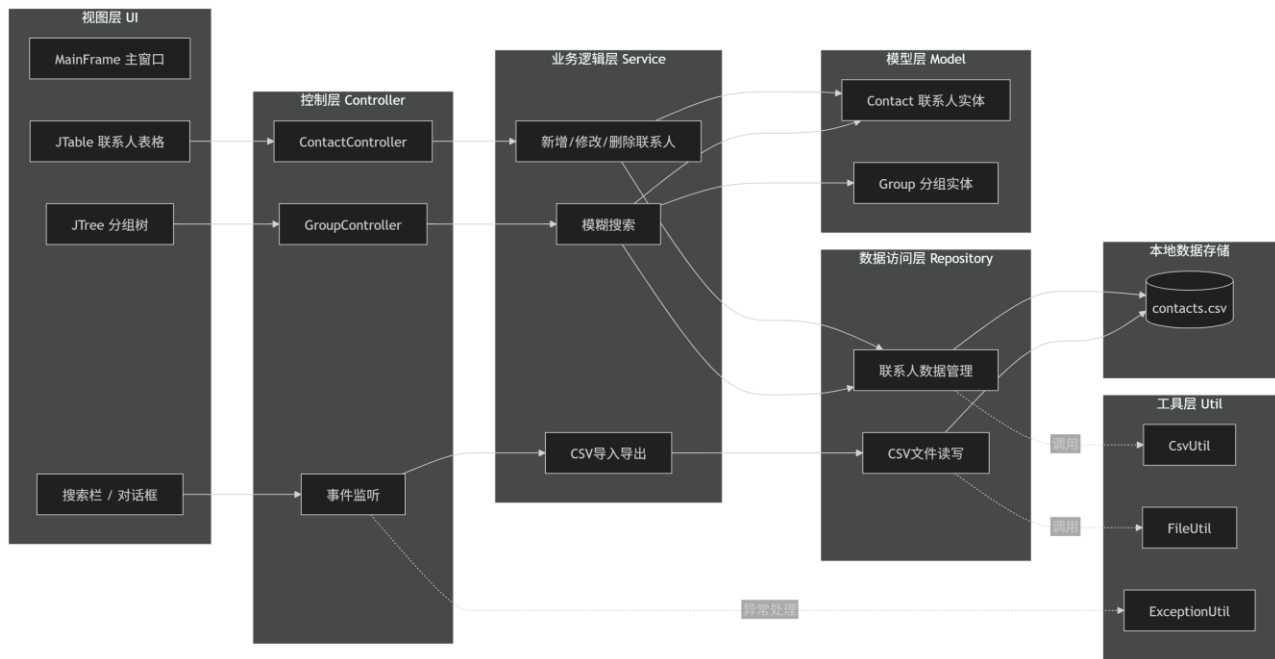
二、系统设计

2.1 系统总体结构设计

2.1.1 系统总体架构

本系统采用 MVC（Model-View-Controller）分层设计思想，并结合 Repository 与 Service 分层结构，实现系统模块之间的低耦合与高内聚，提高系统的可维护性、可扩展性以及代码复用性。

系统整体采用六层结构，包括模型层、数据访问层、业务逻辑层、控制层、视图层以及工具层。各层之间职责明确，通过接口与方法调用完成协同工作。系统总体结构如图所示。



模型层主要用于封装系统中的核心数据对象，即联系人实体类。该层负责定义联系人数据结构，包括联系人姓名、电话号码、邮箱地址、所属分组等属性。模型层不涉及具体业务逻辑与界面显示，仅负责数据的封装与传递，为系统各模块提供统一的数据对象支持。例如：`Contact` 类和 `Group` 类。其主要作用包括：封装联系人基本属性、提供 `Getter` 与 `Setter` 方法、实现对象之间的数据传递、作为业务逻辑处理的数据基础。

数据访问层主要负责系统数据的读取、保存以及文件操作，实现数据持久化功能。本系统采用本地 CSV 文件作为数据存储方式，因此 `Repository` 层负责：联系人数据读取、联系人数据写入、CSV 文件解析、文件创建与更新、数据缓存同步。该层将文件操作与业务逻辑解耦，避免业务代码直接操作文件，提高代码结构清晰度。例如：`ContactRepository`、`CsvRepository`。其核心作用是：实现数据持久化、提供统一的数据访问接口、隔离底层文件操作、提高系统维护性。

业务逻辑层是系统核心功能实现层，主要负责处理联系人管理、分组管理、搜索过滤以及导入导出等业务逻辑。该层接收 `Controller` 层传递的请求，并调用 `Repository` 层完成数据操作，最后将结果返回给界面层。主要功能包括：新增联系人、修改联系人、删除联系人、批量删除、分组筛选、模糊搜索、CSV 导入导出、数据校验。例如：`ContactService`、`GroupService`、`ImportExportService` 业务逻辑层有效实现了系统功能模块化，增强了代码复用能力。

控制层主要负责接收用户界面的事件请求，并协调业务逻辑层与视图层之间的交互。当用户点击按钮、输入数据或执行搜索操作时，`Controller` 会接收对应事件，并调用 `Service` 层

进行业务处理，最终更新界面显示。例如：`ContactController`、`GroupController`。其主要作用包括：监听用户操作、调用业务逻辑处理、控制界面刷新、管理模块间交互。通过 `Controller` 层的引入，系统实现了界面与业务逻辑分离，提高了系统结构规范性。

视图层主要负责系统图形化界面的展示与用户交互，是用户直接操作系统的部分。本系统采用 `Java Swing` 技术实现桌面图形界面，包括：主窗口、联系人表格、分组树、搜索栏、菜单栏、对话框、提示弹窗等。系统通过 `JTable` 展示联系人数据，通过 `JTree` 展示分组结构，使界面更加直观清晰。视图层主要作用包括：展示联系人信息、接收用户输入、提供交互操作入口、提升用户体验。

工具层主要用于存放系统公共工具类与辅助功能，减少重复代码，提高开发效率。工具层主要包括：`CSV` 文件工具、常量管理、输入校验、异常处理、日期处理、`JSON` 扩展预留工具等。例如：`CsvUtil`、`FileUtil`、`ConstantUtil`、`ExceptionUtil`。该层为系统提供基础公共支持，提高整体代码规范性与扩展能力。

2.1.2 系统功能模块设计

为了提高系统功能的模块化程度与代码可维护性，本系统按照功能划分为多个独立模块，各模块之间相互协作，共同完成联系人信息管理任务。系统功能模块主要包括联系人管理模块、分组管理模块、搜索模块、数据持久化模块、`CSV` 导入导出模块以及界面交互模块。

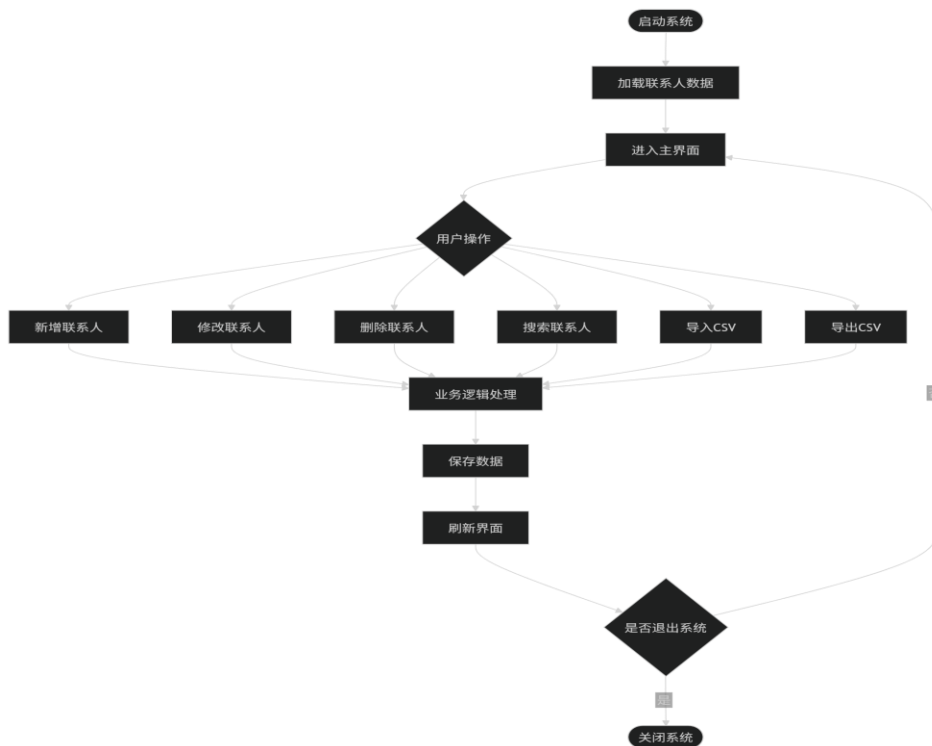
其中，联系人管理模块负责联系人信息的新增、修改、删除及批量删除等基本操作；分组管理模块负责联系人分类管理，实现分组创建、删除以及联系人分组功能；搜索模块支持按照姓名、电话和邮箱等关键字进行模糊查询，提高联系人检索效率；数据持久化模块负责本地文件数据的读取与保存，保证程序关闭后数据不丢失；`CSV` 导入导出模块用于实现联系人数据的批量迁移与备份；界面交互模块则通过表格、树形结构以及提示对话框等方式提升用户使用体验。

2.1.3 系统总体流程设计

系统总体流程主要描述程序从启动到退出过程中各功能模块之间的数据流转关系。系统启动后，首先加载本地存储的联系人数据，并初始化主界面。用户随后可通过图形界面执行新增联系人、修改联系人、删除联系人、搜索联系人以及 `CSV` 导入导出等操作。

当用户发起操作请求后，系统会调用对应的业务逻辑模块进行处理，并将处理结果同步更新到界面中。同时，系统会自动将最新数据保存至本地文件，实现数据持久化。

在整个运行过程中，系统通过事件监听机制实现用户操作与业务逻辑之间的交互，从而保证系统功能的正常运行。系统总体流程如图所示。



2.2 系统各个类及类之间的关系设计

2.2.1 Contact 联系人实体类

`Contact` 类属于系统模型层 (Model)，主要用于封装联系人基础信息，是整个联系人管理系统中的核心实体类。系统在新增、修改、查询以及保存联系人数据时，都会以 `Contact` 对象作为统一的数据载体，实现各模块之间的数据传递与业务处理。

当用户在界面中新增联系人时，系统首先通过无参构造方法或全参构造方法创建 `Contact` 对象，并根据用户输入的信息依次完成姓名、联系电话、即时通讯账号、电子邮箱、出生日、工作单位、居住地址、邮政编码、所属分组以及备注信息等属性的初始化。随后，系统通过各属性对应的 `getter` 与 `setter` 方法实现数据读取与更新，从而完成联系人信息的修改与维护。

在联系人展示或数据导出过程中，系统会调用 `toString()` 方法，将联系人对象中的各项属性按照统一格式输出，便于日志打印、调试以及联系人信息查看。同时，为了实现联系人数据的本地持久化存储，`Contact` 类实现了 `Serializable` 序列化接口，使系统能够将联系人对象整体写入本地文件，并在程序重新启动后自动恢复联系人数据，从而保证数据的完整性与连续性。

2.2.2 ContactRepository 联系人数据仓库类

`ContactRepository` 类属于数据访问层 (Repository)，主要负责联系人数据的本地持久化管理，实现联系人数据与磁盘文件之间的读写交互。系统启动时，程序首先调用 `loadContactData()` 方法读取本地 `contacts.data` 文件，并通过反序列化方式将文件中的联系人数据恢复为内存中的联系人集合，完成系统数据初始化。当用户新增、修改或删除联系人后，系统会自动调用 `saveContactData()` 方法，将当前最新联系人集合重新序列化写入本地文件，从而保证数据实时保存。为了支持界面刷新与数据重载，类中还提供 `clearContactCache()` 方法，用于清空当前缓存联系人集合，避免旧数据残留。通过该类的封装，系统实现了业务逻辑与底层 IO 操作分离，提高了整体代码结构清晰度与可维护性。

2.2.3 GroupRepository 分组数据仓库类

GroupRepository 类属于数据访问层（Repository），主要用于独立管理系统中的所有联系人分组数据，实现分组信息的单独持久化存储。系统启动后，会首先调用 loadGroupData() 方法读取 groups.data 文件，将系统已有分组加载至内存中的 allGroupSet 集合中。当用户新增或删除分组时，系统会同步调用 saveGroupData() 方法，将更新后的分组集合重新写入本地文件，保证分组数据与系统状态保持一致。在界面需要刷新分组树或获取全部分组信息时，可通过 getAllGroup() 方法统一获取当前系统中的所有分组数据。通过将分组管理与联系人数据管理分离，系统降低了模块耦合度，提高了后期扩展能力。

2.2.4 SearchService 搜索筛选服务类

SearchService 类属于业务逻辑层（Service），主要负责系统中的联系人搜索与数据筛选功能。用户在搜索框输入关键字后，系统会调用 fuzzySearch() 方法，对联系人姓名、电话号码以及邮箱等字段进行模糊匹配，并自动忽略英文字母大小写，从而提高搜索灵活性。当用户点击左侧分组树时，系统则通过 filterByGroup() 方法筛选出属于指定分组的联系人数据，实现精准分类查看。如果用户清空搜索条件或需要恢复全部联系人列表，系统会调用 getAllContact() 方法重新获取完整联系人数据，并刷新表格显示。该类将搜索逻辑独立封装，避免与联系人增删改业务混杂，提高了业务模块划分的合理性。

2.2.5 ContactService 联系人业务服务类

ContactService 类属于业务逻辑层（Service），是系统中最核心的业务处理类，主要负责联系人数据的新增、修改、删除以及批量操作等业务逻辑处理。用户在界面中提交新增联系人信息后，系统会首先调用 addContact() 方法，对联系人姓名与手机号等关键数据进行合法性校验，例如判断手机号是否为空、联系人姓名是否重复等，校验通过后再将联系人加入集合，并自动调用数据仓库完成本地保存。当用户修改联系人信息时，系统会通过 updateContact() 方法使用新对象覆盖原有联系人数据，并同步更新文件内容。对于删除操作，系统通过 deleteContact() 删除指定联系人，而 batchDeleteContact() 方法则采用倒序删除方式批量移除联系人，避免集合索引变化导致删除错误。界面在刷新联系人列表时，可通过 getContactList() 方法统一获取全部联系人数据。该类实现了联系人核心业务逻辑与数据操作之间的统一调度。

2.2.6 GroupService 分组业务服务类

GroupService 类属于业务逻辑层（Service），主要负责联系人分组的全生命周期管理，包括分组创建、删除以及联系人与分组之间的绑定关系维护。当用户创建新分组时，系统会调用 addGroup() 方法自动检测分组名称是否重复，若不存在相同分组，则将其加入系统分组集合并保存至本地文件。当用户删除分组时，deleteGroup() 方法不仅会移除系统中的该分组，还会同步遍历所有联系人，将联系人身上的对应分组标签一并清除，保证数据一致性。在联系人管理过程中，系统还可以通过 addContactToGroup() 方法将联系人加入指定分组，或通过 removeContactFromGroup() 方法移除联系人已有分组关系。界面在刷新分组树时，则通过 getAllSystemGroup() 获取系统当前全部分组信息，实现分组数据与界面同步更新。

2.2.7 CsvUtil CSV 文件工具类

CsvUtil 类属于工具层（Util），主要用于实现联系人数据的 CSV 文件导入与导出功能。用户执行数据导出操作时，系统会调用 exportToCsv() 方法，将当前联系人列表按照固定表头格式写入指定 CSV 文件，实现联系人数据备份与跨平台迁移。当用户选择 CSV 文件进

行导入时，系统首先调用 `importFromCsv()` 方法读取文件内容，并逐行解析联系人数据，将每条数据重新封装为 `Contact` 对象，同时自动识别文件中的新分组并同步创建。在解析联系人所属分组字段时，系统会进一步调用 `splitGroupStr()` 方法，将 CSV 文件中的多分组字符串拆分为程序可识别的分组集合。通过该工具类，系统实现了联系人数据的批量导入导出，提高了数据迁移效率。

2.2.8 ErrorHandler 全局异常处理类

`ErrorHandler` 类属于工具层（Util），主要用于统一处理系统运行过程中产生的各类异常信息，避免程序因异常直接崩溃。当系统在读取文件、保存数据或执行序列化操作过程中发生 IO 异常时，程序会调用 `handleIOException()` 方法进行统一处理，并通过提示框向用户显示错误信息。当用户输入空数据或程序出现空对象访问情况时，系统则调用 `handleNullException()` 方法进行异常拦截与提示。此外，系统中的各类成功、失败以及警告提示，均通过 `showTipDialog()` 方法统一弹出对应类型的提示窗口，从而增强用户交互体验，提高系统运行稳定性。

2.2.9 Constants 全局常量工具类

`Constants` 类属于工具层（Util），主要用于集中管理系统中的公共常量数据，避免程序中大量出现硬编码问题。系统运行过程中涉及的数据文件路径、默认分组名称、CSV 文件表头字段以及界面提示文字等固定内容，均统一存放于该类中。当系统读取本地文件或创建默认分组时，会直接调用对应常量数据，从而提高代码统一性与后期维护效率。通过集中管理常量，开发人员在修改系统配置或调整界面提示内容时，仅需修改对应常量即可完成全局更新。

2.2.10 AppController 全局总控制器

`AppController` 类属于控制层（Controller），主要负责协调整个系统中各业务模块之间的交互，是界面层与业务逻辑层之间的核心调度中心。系统启动后，`AppController` 会统一初始化联系人服务、分组服务以及搜索服务等核心业务对象。当用户在界面中执行新增联系人、删除分组或搜索联系人等操作时，系统会先将请求发送至 `AppController`，再由其根据操作类型分发给对应子控制器处理。业务执行完成后，`AppController` 会统一接收处理结果，并通知界面刷新对应数据，实现系统整体流程协调控制。

2.2.11 ContactController 联系人控制器

`ContactController` 类属于控制层（Controller），主要负责联系人相关界面事件的监听与处理。当用户点击新增联系人按钮时，控制器会接收界面输入数据，并调用 `ContactService` 完成联系人创建与保存；当用户修改联系人信息时，系统会先读取当前联系人数据，再调用更新业务完成数据替换；对于单条删除与批量删除操作，控制器会根据用户选中的联系人下标调用对应删除方法，并在操作完成后刷新联系人表格数据。此外，该控制器还负责联系人数据刷新与界面同步更新，实现界面与业务逻辑之间的数据交互。

2.2.12 GroupController 分组控制器

`GroupController` 类属于控制层（Controller），主要负责分组相关界面操作事件的处理。当用户在左侧分组树中新建分组时，系统会通过控制器调用 `GroupService` 完成分组创建与保存；删除分组时，控制器则会同步更新联系人中的分组信息，并刷新分组树结构。在联系人分组管理过程中，用户还可以通过界面操作将联系人加入指定分组或移出已有分组，而这

些操作均由 GroupController 负责接收并转发给对应业务逻辑处理。通过该类的封装，系统实现了分组界面与业务逻辑之间的有效解耦。

2.2.13 MainFrame 系统主窗口界面类

MainFrame 类属于视图层（UI），是整个联系人管理系统的主窗口界面，也是系统图形界面的入口。系统启动后，程序首先创建 MainFrame 窗口，并完成顶部功能按钮栏、左侧分组树区域、中间联系人表格区域以及底部搜索操作栏等主要界面布局初始化。在界面加载完成后，系统会进一步绑定新增联系人、删除联系人、导入导出以及搜索等按钮事件，实现用户操作与控制器之间的交互。同时，该类还统一负责窗口大小管理、关闭事件监听以及程序退出时的数据保存操作，是整个系统界面运行的核心容器。

2.2.14 GroupTreePanel 分组树面板类

GroupTreePanel 类属于视图层（UI），主要负责左侧分组树形菜单的界面展示与交互功能。系统初始化时，会首先读取当前所有联系人分组，并通过 JTree 树形组件构建分组层级结构。当用户点击不同分组节点时，系统会自动触发分组筛选操作，仅显示属于当前分组的联系人数据。在分组新增或删除后，系统还会调用 refreshGroupTree() 方法重新加载分组数据并刷新树形菜单结构，保证界面显示与系统数据保持一致。此外，该类还集成了分组右键菜单功能，方便用户快速执行分组管理操作。

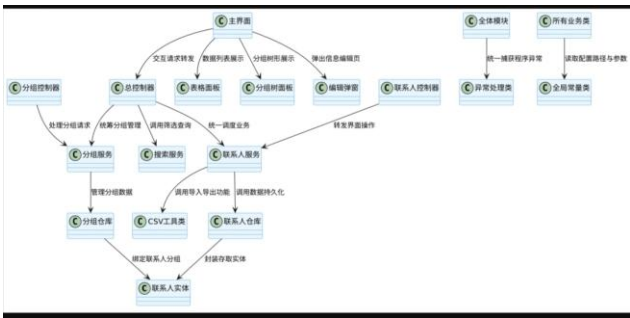
2.2.15 ContactTablePanel 联系人表格面板类

ContactTablePanel 类属于视图层（UI），主要用于实现联系人数据表格的展示与操作。系统在加载联系人数据后，会调用 setTableData() 方法将联系人列表批量填充到 JTable 表格中，实现联系人信息可视化展示。当用户执行批量删除操作时，系统则通过 getCheckedRows() 方法获取当前被勾选的联系人行下标，并将其传递给业务层进行批量删除处理。在联系人数据刷新或搜索条件变化时，程序还会调用 clearTableData() 方法清空原有表格数据，再重新加载最新联系人列表，从而保证界面数据实时更新。

2.2.16 ContactDialog 联系人编辑弹窗类

ContactDialog 类属于视图层（UI），主要用于统一封装新增联系人与修改联系人所使用的弹窗界面。用户点击新增联系人按钮后，系统会调用 showAddDialog() 方法初始化空白表单，并显示联系人信息输入界面；当用户修改已有联系人时，程序则通过 showUpdateDialog() 方法将原有联系人数据自动回填至输入框中，方便用户直接编辑已有信息。在用户完成信息填写并点击确认按钮后，系统会调用 getInputContact() 方法读取所有表单输入内容，并自动封装为完整的 Contact 实体对象，随后交由控制层与业务层完成后续处理。

2.2.17 类之间的关系图



2.3 数据存储的设计

2.3.1 整体存储方案设计

本通讯录管理系统不依赖数据库，采用本地文件持久化存储方案，结合 Java 对象序列化与 CSV 文件辅助存储两种方式，分别用于程序内部数据永久保存和跨设备数据导入导出备份。整体设计遵循结构独立、分层存储、格式规范、自动读写的原则，保证程序关闭后数据不丢失、重启自动加载，同时支持数据迁移与批量备份。

2.3.2 持久化文件规划与作用

为了保证系统数据在程序关闭后不会丢失，本系统采用本地文件持久化存储方式，并使用“双文件分离存储”结构，将联系人数据与分组数据分别独立保存。该设计能够有效降低数据之间的耦合度，避免联系人信息与分组信息混杂，同时提高系统后期维护与扩展能力。

系统启动时，程序会自动检测本地数据文件是否存在；若文件存在，则自动读取并恢复系统数据；若文件不存在，则由系统自动创建对应数据文件。用户在新增、修改或删除联系人及分组后，系统会同步更新本地文件，实现数据实时保存。

其中，`contacts.data` 文件主要用于保存系统中的全部联系人信息。该文件会自动生成在项目根目录下，并采用二进制对象序列化方式存储联系人集合 `List`。系统在运行过程中，会将联系人姓名、联系电话、电子邮箱、工作单位、居住地址、生日信息、所属分组以及备注等完整数据统一封装为 `Contact` 对象，并整体写入文件中。由于采用序列化存储方式，用户无需手动编辑文件内容，所有读写操作均由程序自动完成，从而提高了数据安全性与存储效率。

`groups.data` 文件则用于独立保存系统中的所有联系人分组信息，同样存放于项目根目录下。系统会通过序列化方式将全部分组名称封装为 `Set` 集合并写入文件，包括系统默认分组以及用户后续自定义创建的分组内容。当用户新增或删除分组时，程序会自动同步更新该文件。通过将分组数据单独存储，系统能够更加清晰地管理联系人分类结构，同时避免联系人数据与分组数据之间产生过度依赖。

2.3.3 内存数据结构设计

为了提高系统运行过程中联系人数据的管理效率，本系统在内存中采用不同类型的集合结构对联系人与分组数据进行统一管理，并根据各模块业务特点选择合适的数据结构，以提升数据操作性能与程序运行稳定性。

在联系人数据管理方面，系统使用 `List<Contact>` 集合存储全部联系人对象。程序启动后，会先从本地文件中读取联系人数据，并加载到联系人集合中。由于 `List` 集合能够保持数据插入顺序，因此系统可以按照联系人添加顺序稳定展示数据，便于 `JTable` 表格组件进行行数据渲染。同时，`List` 集合支持通过下标快速定位指定联系人，因此在修改联系人、删除联系人以及批量删除联系人时，系统能够直接根据表格选中行对应的下标完成数据操作。此外，`List` 结构还能够灵活支持联系人新增、遍历搜索以及数据刷新等业务需求，较适合通讯录系统中的动态数据管理场景。

在分组数据管理方面，系统使用 `Set<String>` 集合存储所有分组名称。系统初始化时，会将本地文件中的全部分组数据加载至集合中；当用户新增分组时，程序会首先判断当前集合中是否已存在相同分组名称。由于 `Set` 集合具有元素自动去重特性，因此能够有效避免用户创建重复分组，保证系统分组名称唯一性。同时，`Set` 集合在判断元素是否存在时效率较高，适用于系统中的分组重复检测与快速查询操作。通过使用独立的 `Set` 集合管理分组数据，系统进一步提高了分组管理逻辑的规范性与运行效率。

2.3.4 序列化存储机制设计

为了保证联系人数据在程序关闭后仍能够长期保存，本系统采用 Java 对象序列化机制实现本地数据持久化存储。系统首先让 `Contact` 实体类实现 `Serializable` 序列化接口，从而使联系人对象具备对象流读写能力。在数据保存过程中，程序会通过 `ObjectOutputStream` 输出流，将内存中的联系人集合与分组集合整体转换为二进制数据，并写入本地 `.data` 文件中；当系统重新启动时，则通过 `ObjectInputStream` 输入流读取文件内容，并将二进制数据重新反序列化为程序中的对象集合，从而恢复系统运行所需的数据状态。

在程序运行流程中，系统对数据读写时机进行了统一规划。系统启动初始化阶段，会首先自动检测本地数据文件是否存在，并读取联系人与分组数据加载至内存集合中，完成系统数据恢复。当用户执行新增联系人、修改联系人、删除联系人、新增分组、删除分组以及批量删除等操作后，系统会自动触发保存机制，将当前最新数据实时同步到本地文件，避免因程序异常关闭导致数据丢失。整个过程无需用户手动保存，提升了系统使用便捷性。

为了提高系统运行稳定性，本系统还设计了基础容错处理机制。当程序启动时，如果本地数据文件不存在、文件内容损坏或读取过程中发生异常，系统不会直接报错或终止运行，而是自动创建空集合继续初始化程序，并通过异常处理模块输出友好提示信息。通过该设计，即使本地数据文件出现问题，系统仍能够正常启动与运行，从而提高了程序的健壮性与用户体验。

2.3.5 CSV 导入导出存储格式设计

为了满足联系人数据备份、跨设备迁移以及批量导入录入等实际使用需求，本系统设计了统一规范的 CSV 数据交换格式，实现联系人数据的批量导入与导出功能。CSV 文件具有结构简单、兼容性强以及能够直接使用 Excel 打开的特点，因此适合作为系统数据交换与备份格式。

系统首先对 CSV 文件表头进行了统一规范设计，所有联系人数据均按照“姓名、电话、即时通讯、邮箱、生日、单位、地址、邮编、分组、备注”的固定字段顺序进行存储。程序在导出数据时，会严格按照该字段顺序逐行写入联系人信息，从而保证导入与导出格式一致，避免字段错位问题。

在字段存储规则方面，系统采用逗号作为字段分隔符，每个联系人对应 CSV 文件中的一行数据。当单个联系人同时属于多个分组时，系统会将多个分组名称拼接后统一存储于“分组”字段中，从而实现多分组信息保存。同时，为避免中文姓名、地址或备注信息出现乱码问题，系统统一采用 UTF-8 编码格式读写 CSV 文件，提高文件跨平台兼容性。

在导出流程中，系统会首先遍历内存中的联系人集合，然后按照统一字段格式逐行生成 CSV 数据，并写入指定文件中。用户导出的 CSV 文件不仅可用于系统数据备份，还能够直接通过 Excel 等办公软件打开、查看与编辑。

在导入流程中，系统会逐行读取 CSV 文件内容，并按照字段顺序自动解析联系人信息，随后封装为 `Contact` 对象加入系统联系人集合。在解析联系人所属分组时，如果系统发现 CSV 文件中存在当前系统尚未创建的新分组，则会自动完成分组创建，无需用户提前手动添加分组，从而提高了批量导入操作的便捷性与智能化程度。

2.4 界面设计

2.4.1 整体界面布局

系统基于 Java Swing 开发，采用顶部功能栏 + 左侧分组树 + 中间数据表格 + 底部搜索栏经典桌面布局，界面分区清晰、结构规整，符合常规软件操作习惯，布局简洁美观、交互直观。

2.4.2 主界面设计

顶部功能按钮区集中放置新增、修改、删除、批量操作、分组管理、CSV 导入导出等核心功能按钮，操作入口集中，可快速完成常用功能调用。

左侧分组树区域使用 JTree 树形组件展示分组，内置所有联系人、家人、朋友、同学、同事默认分组，同时支持自定义新增、删除分组。点击对应分组可实时筛选表格联系人，实现分类快速查看。

中间联系人表格区域采用 JTable 组件展示联系人数据，包含复选框、姓名、电话、邮箱、分组、备注等列。支持单行选中、全选批量勾选，便于批量删除和批量分组；表格采用斑马线样式、选中行高亮，提升阅读体验，数据更新后自动刷新。

底部搜索区域设置关键词输入框、搜索按钮与重置按钮，支持按姓名、电话、邮箱模糊检索；清空关键词可重置展示全部联系人，方便快速查找数据。

2.4.3 编辑弹窗设计

设计统一的联系人编辑弹窗，复用实现新增和修改功能。弹窗采用表单布局，包含姓名、电话、邮箱、单位、地址、生日、备注等输入项，并提供分组多选复选框。新增时清空表单，修改时自动回填原有数据，操作便捷统一。

2.4.4 交互与视觉设计规范

按钮按功能配色区分：绿色新增、蓝色常规操作、红色删除操作，视觉辨识度高。

所有操作执行后弹出统一风格提示弹窗，对成功、失败、异常情况给出友好文字提示，避免程序闪退。

主窗口及所有子弹窗均居中显示，组件布局规整，Windows 系统下无乱码、无界面错位，运行稳定。

主界面（MainFrame）

顶部：功能按钮栏（新增、修改、删除、分组、导入导出等）。

左侧：分组树（所有联系人、家人、朋友、同学、同事及自定义分组）。

中间：联系人表格（复选框、姓名、电话、邮箱、分组、备注）。

底部：搜索框、搜索/重置按钮。

编辑对话框（ContactDialog）

表单输入：姓名、电话、邮箱、地址、邮编、生日、单位、备注。

分组多选：JCheckBox 动态加载分组。

交互规范

按钮按功能配色：绿色新增、蓝色常规、红色删除、黄色警告。

表格斑马线、选中高亮、提示弹窗统一风格。

三、系统实现

3.1 核心功能实现

本系统基于 Java Swing 开发，采用 MVC 分层思想，将界面层、控制层、业务层、数据层分离。主要实现：联系人增删改查、分组管理、模糊搜索、数据持久化、CSV 导入导出、全局异常弹窗提示等功能

3.1.1 联系人实体类实现

功能说明

设计联系人实体类，封装姓名、电话、邮箱、单位、地址、分组列表、备注等属性；实现 Serializable 序列化接口，支持对象本地文件存储；提供构造方法与 get/set 方法，统一数

据封装格式。

核心代码片段

```
public class Contact implements Serializable {  
    private String name;  
    private String phone;  
    private String email;  
    private String company;  
    private List<String> groupList;  
    private String remark; // 构造方法、getter、setter 省略}
```

3.1.2 数据持久化功能实现

功能说明

采用 Java 对象序列化技术，自定义数据仓库类。程序启动自动反序列化加载联系人、分组数据；进行新增、修改、删除操作后自动序列化保存到本地 .data 文件，实现关闭程序数据不丢失、重启自动恢复。若文件不存在或损坏，自动初始化空集合，保证程序正常运行

核心代码片段

```
// 加载数据  
public void loadData() throws Exception {  
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream("contacts.data"));  
    contactList = (List<Contact>) ois.readObject();  
}  
  
// 保存数据  
public void saveData() throws Exception {  
    ObjectOutputStream oos = new ObjectOutputStream(new  
        FileOutputStream("contacts.data"));  
    oos.writeObject(contactList);  
}
```

3.1.3 联系人新增功能实现

功能说明

接收界面表单输入数据，封装为 Contact 对象；业务层校验姓名非空、姓名不重复，校验通过后加入集合，并自动调用持久化方法保存文件，同时刷新界面表格。

核心代码片段

```
public boolean addContact(Contact contact) {  
    // 姓名重复校验  
    for (Contact c : contactList) {  
        if (c.getName().equals(contact.getName())) {  
            return false;  
        }  
    }  
}
```

```
contactList.add(contact);
repository.saveData();
return true;
}
```

3.1.4 联系人修改功能实现

功能说明

获取表格选中行下标，将编辑弹窗回填原数据；修改提交后，替换集合中对应下标对象，自动保存并刷新界面，完成信息更新。

核心代码片段

```
public boolean updateContact(int index, Contact newContact) {
    if (index < 0 || index >= contactList.size()) {
        return false;
    }
    contactList.set(index, newContact);
    repository.saveData();
    return true;
}
```

3.1.5 联系人删除及批量删除实现

功能说明

支持单条删除与批量勾选删除；批量删除采用倒序遍历删除，避免正序删除导致集合下标错乱；删除后自动保存并刷新表格。

核心代码片段

```
// 单条删除
public boolean deleteContact(int index) {
    contactList.remove(index);
    repository.saveData();
    return true;}
// 批量倒序删除
public void batchDelete(List<Integer> idxList) {
    for (int i = idxList.size() - 1; i >= 0; i--) { contactList.remove(idxList.get(i));
    }
    repository.saveData();
}
```

3.1.6 分组管理功能实现

功能说明

支持新增自定义分组、删除分组；删除分组时自动遍历所有联系人，清除该分组关联标签；分组数据独立持久化，左侧分组树实时刷新。

核心代码片段

```

// 新增分组
public boolean addGroup(String groupName) {
    if (allGroup.contains(groupName)) return false;
    allGroup.add(groupName);
    groupRepo.saveGroupData();
    return true;
}
// 删除分组并清理联系人关联
public void deleteGroup(String groupName) {
    allGroup.remove(groupName);
    for (Contact c : contactList) {
        c.getGroupList().remove(groupName);
    }
    groupRepo.saveGroupData();
}

```

3.1.7 模糊搜索与分组筛选实现

功能说明

利用 Java Stream 流式编程，实现姓名、电话、邮箱多字段模糊匹配；支持按分组名称过滤联系人，筛选结果实时刷新表格，清空关键词恢复全部数据。

核心代码片段

```

// 模糊搜索
public List<Contact>
fuzzySearch(List<Contact> list, String key) {
    return list.stream()
        .filter(c -> c.getName().contains(key)
            || c.getPhone().contains(key)
            || c.getEmail().contains(key))
        .collect(Collectors.toList());
}
// 按分组筛选
public List<Contact>
filterByGroup(List<Contact> list, String group) {
    return list.stream()
        .filter(c ->
            c.getGroupList().contains(group))
        .collect(Collectors.toList());
}

```

3.1.8 CSV 导入导出功能实现

功能说明

封装 CSV 工具类，可将联系人批量导出为标准 CSV 文件，支持 Excel 打开备份；也可读取外部 CSV 文件，逐行解析自动封装为联系人对象，批量导入系统并自动创建新分组。

核心代码片段

```
// CSV 导出
public static void exportCsv(List<Contact> list, String path) throws IOException {
    BufferedWriter bw = new BufferedWriter(new FileWriter(path));
    bw.write("姓名,电话,邮箱,分组,备注");
    for (Contact c : list) {
        bw.write(c.getName()+","+c.getPhone()+","+c.getEmail()); bw.newLine();
    }
    bw.close();
}
```

3.1.9 全局异常与弹窗提示实现

功能说明 统一封装异常处理工具类，捕获 IO 异常、空指针、非法输入等问题；通过 Swing 弹窗给出成功、错误、警告友好提示，不暴露底层异常，防止程序闪退。

核心代码片段

```
public class ErrorHandler {
    // 成功提示
    public static void showSuccess(String msg) {
        JOptionPane.showMessageDialog(null, msg, "提示",
        JOptionPane.INFORMATION_MESSAGE);
    }
    // 错误提示
    public static void showError(String msg) {
        JOptionPane.showMessageDialog(null, msg, "错误", JOptionPane.ERROR_MESSAGE);
    }
}
```

四、系统测试

4.1 模块测试

模块测试针对系统各核心功能模块单独开展测试，验证每个模块的功能正确性、数据合法性校验及异常处理能力，测试步骤统一为：①设计测试用例（输入数据+预期结果）；②实际运行程序获取运行结果；③对比分析得到测试结论。

4.1.1 联系人管理模块测试

测试用例 1：新增联系人（正常输入）

- ① 设计测试用例：输入数据（姓名：张三，电话：13800138000，邮箱：zhangsan@163.com，分组：朋友）；预期结果：联系人新增成功，表格显示该联系人，本地文件同步保存。
- ② 实际运行结果：点击“新增联系人”，填写上述信息提交，弹出“新增成功”提示，联系人表格同步显示该条数据，关闭程序重启后，数据依然存在。
- ③ 测试结论：新增联系人功能正常，数据持久化有效，符合预期。

测试用例 2：新增联系人（重名输入）

- ① 设计测试用例：输入数据(姓名:张三,电话:13900139000,邮箱:zhangsan123@163.com);预期结果：提示“姓名重复,无法新增”,联系人未添加。
- ② 实际运行结果：填写重名信息提交,弹出“姓名重复,无法新增”提示,表格无新增数据,本地文件未更新。
- ③ 测试结论：重名校验功能有效,符合预期。

测试用例 3: 修改联系人

- ① 设计测试用例：选中“张三”，输入修改数据（电话：13811138111，邮箱：zhangsan_new@163.com）；预期结果：修改成功，表格显示更新后信息，本地文件同步更新。
- ② 实际运行结果：提交修改后，弹出“修改成功”提示，表格中“张三”的电话、邮箱同步更新，重启程序后，修改后的数据正常显示。
- ③ 测试结论：修改联系人功能正常，数据同步有效。

测试用例 4: 删除联系人（单个+批量）

- ① 设计测试用例：单个删除（选中“张三”，点击删除）；批量删除（选中“张三”“李四”，点击批量删除）；预期结果：删除成功，表格中对应联系人消失，本地文件同步删除。
- ② 实际运行结果：单个删除后弹出“删除成功”提示，表格中“张三”消失；批量删除后，选中的两个联系人均消失，重启程序无残留数据。
- ③ 测试结论：单个、批量删除功能正常，无索引错乱问题，符合预期。

4.1.2 分组管理模块测试

测试用例 1: 新增分组

- ① 设计测试用例：输入分组名称“同事”；预期结果：分组新增成功，左侧分组树显示“同事”，本地分组文件同步保存。
- ② 实际运行结果：提交分组名称后，左侧分组树实时刷新，显示“同事”分组，重启程序后分组依然存在。
- ③ 测试结论：新增分组功能正常，数据持久化有效。

测试用例 2: 删除分组

- ① 设计测试用例：选中分组“同事”，点击删除；预期结果：分组删除成功，分组树中“同事”消失，绑定该分组的联系人自动解除关联。
- ② 实际运行结果：删除后分组树中“同事”消失，此前绑定“同事”分组的联系人，其分组列表中已无“同事”选项，无无效数据残留。
- ③ 测试结论：删除分组功能正常，关联数据清理彻底。

测试用例 3: 联系人分组绑定/解绑

- ① 设计测试用例：选中联系人“李四”，将其加入“同事”分组；再将其移出“同事”分组；预期结果：加入后分组列表显示“同事”，移出后不再显示。
- ② 实际运行结果：加入分组后，“李四”的分组信息更新为“同事”；移出后，分组信息中无“同事”，分组筛选时可正常匹配/排除该联系人。
- ③ 测试结论：分组绑定、解绑功能正常，筛选逻辑无误。

4.1.3 搜索与筛选模块测试

测试用例 1: 模糊搜索

- ① 设计测试用例：输入关键词“张”；预期结果：搜索出所有姓名包含“张”的联系人（如张三、张五）。

- ② 实际运行结果：输入“张”后，表格实时刷新，仅显示姓名包含“张”的联系人，无遗漏、无错误匹配。
- ③ 测试结论：模糊搜索功能正常，多字段（姓名、电话）匹配有效。

测试用例 2：分组筛选

- ① 设计测试用例：点击分组树“朋友”；预期结果：表格仅显示绑定“朋友”分组的联系人。
- ② 实际运行结果：点击“朋友”分组后，表格实时筛选，仅显示属于该分组的联系人，切换分组后筛选结果同步更新。
- ③ 测试结论：分组筛选功能正常，筛选结果精准。

4.1.4 CSV 导入导出模块测试

测试用例 1：CSV 导出

- ① 设计测试用例：点击“导出 CSV”，选择保存路径；预期结果：导出 CSV 文件，包含所有联系人信息，表头完整、格式规范。
- ② 实际运行结果：导出后用 Excel 打开，表头为“姓名,电话,邮箱,单位,地址,分组,备注”，所有联系人数据完整，无缺失、无乱码。
- ③ 测试结论：CSV 导出功能正常，数据格式符合要求。

测试用例 2：CSV 导入

- ① 设计测试用例：导入提前准备的 CSV 文件（包含 2 条新联系人：王五、赵六）；预期结果：批量导入成功，表格显示新增联系人，新增分组自动创建。
- ② 实际运行结果：导入后弹出“导入成功”提示，表格新增王五、赵六两条数据，CSV 中包含的新分组“同学”自动添加到分组树。
- ③ 测试结论：CSV 导入功能正常，支持批量录入和新增分组自动创建。

4.1.5 数据持久化模块测试

测试用例：数据保存与加载

- ① 设计测试用例：新增联系人“孙七”，修改“王五”的电话，删除“赵六”，关闭程序后重启；预期结果：重启后，新增、修改、删除的数据均生效，无数据丢失。
- ② 实际运行结果：重启程序后，“孙七”正常显示，“王五”的电话为修改后内容，“赵六”已消失，所有操作均同步保存。
- ③ 测试结论：数据持久化功能正常，程序重启后数据不丢失、不错乱。

4.2 系统测试

4.2.1 测试用例：系统完整功能流程测试

- ① 设计测试用例：输入数据（启动程序→新增联系人“周八”→将其加入“同学”分组→搜索关键词“周”→修改“周八”的邮箱→批量删除“周八”“王五”→导出 CSV→导入 CSV→关闭程序重启）；

预期结果：全流程操作正常，数据同步无误，程序无闪退、无异常。

- ② 实际运行结果：启动程序后，依次完成所有操作，每一步均弹出对应成功提示，表格、分组树、本地文件数据同步更新；导出的 CSV 文件数据完整，导入后新增数据正常显示；重启程序后，所有操作结果均保留，程序运行流畅，无卡顿、闪退。

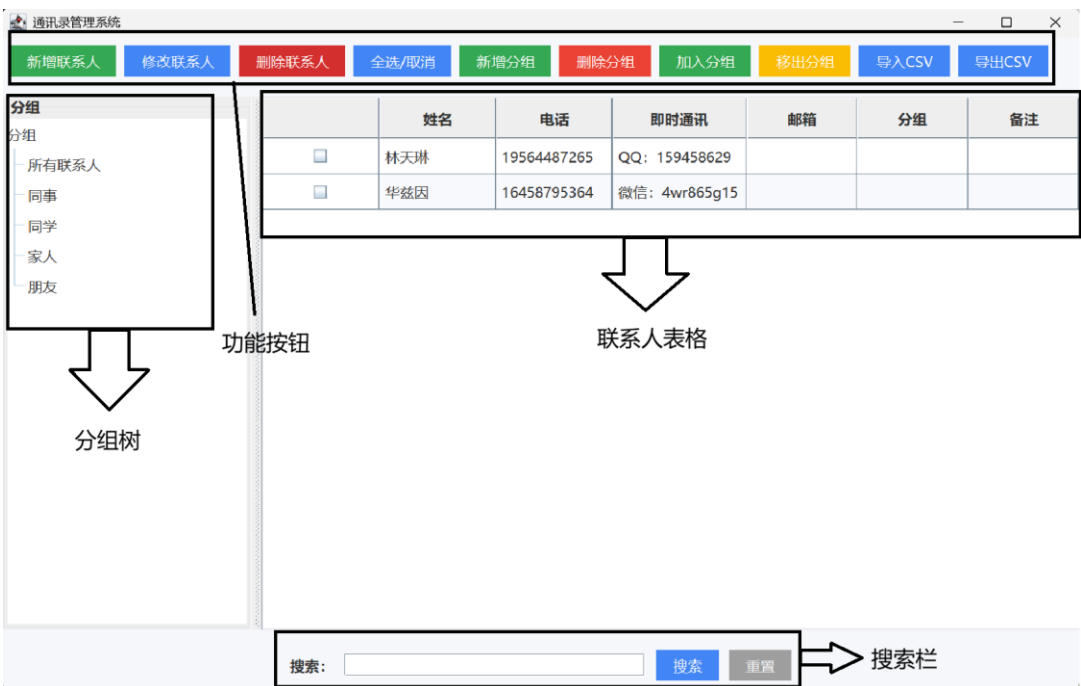
- ③ 测试结论：系统完整功能流程正常，各模块协同工作良好，整体运行稳定。

4.2.2 测试用例：异常场景测试

- ① 设计测试用例：输入数据（空姓名新增、非法手机号输入、删除不存在的联系人、导入格式错误的 CSV 文件、清空搜索关键词）；预期结果：程序弹出友好提示，不闪退，无异常报错。
- ② 实际运行结果：空姓名新增弹出“姓名不能为空”，非法手机号弹出“手机号格式错误”，删除不存在联系人弹出“联系人不存在”，导入错误 CSV 弹出“文件格式错误”，清空搜索关键词显示全部联系人；所有异常场景均有提示，程序正常运行，无闪退。
- ③ 测试结论：系统异常处理能力良好，能有效拦截非法操作，运行稳定。

五、系统运行界面

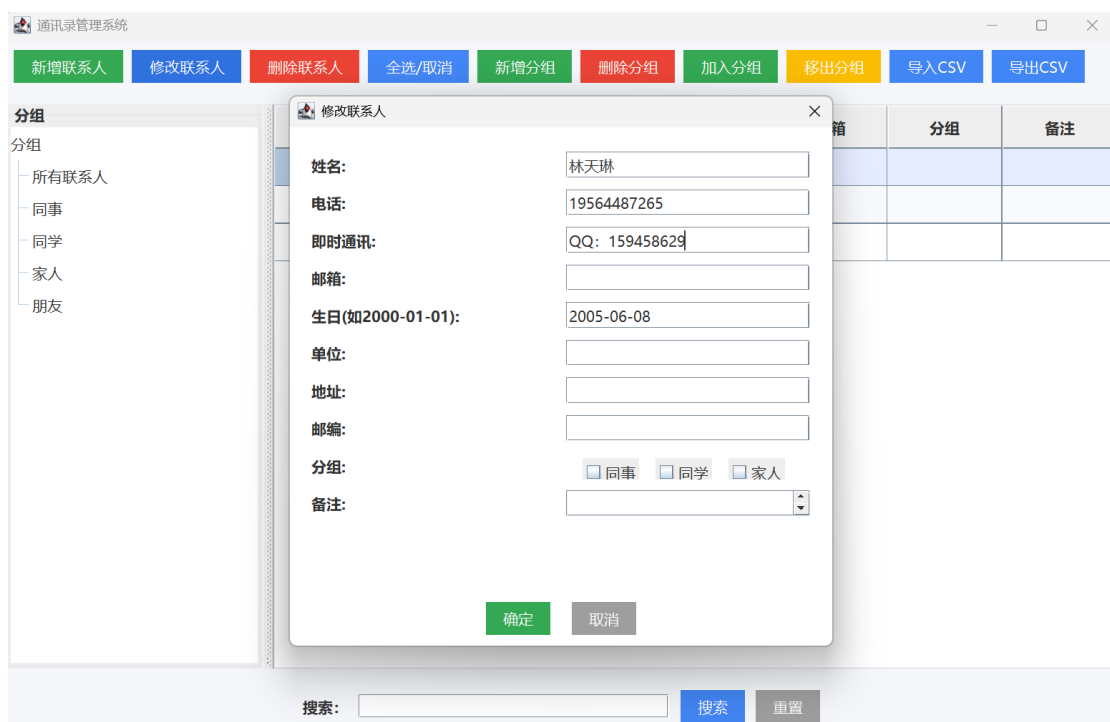
5.1 主界面



5.2 新增/修改联系人



修改联系人①（按顺序单击两个按键）



弹出页面后按需修改即可

5.3 分组管理



① 新增分组（需与已有分组不重名）



② 删除分组（按顺序依次单击删除想要的分组）



③加入分组（按顺序一次操作，多选时使用拖动或勾选二选一）



④移除分组（和③操作类似）

5.4 搜索结果

通讯录管理系统

新增联系人

修改联系人

删除联系人

全选/取消

新增分组

删除分组

加入分组

移出分组

导入CSV

导出CSV

分组

所有联系人

同事

同学

家人

朋友

	姓名	电话	即时通讯	邮箱	分组	备注
☐	田光	13659846859	QQ: 1255546895			

搜索:

搜索重置

可针对名字进行模糊搜索

通讯录管理系统

新增联系人

修改联系人

删除联系人

全选/取消

新增分组

删除分组

加入分组

移出分组

导入CSV

导出CSV

分组

所有联系人

同事

同学

家人

朋友

	姓名	电话	即时通讯	邮箱	分组	备注
☐	林天琳	19564487265	QQ: 159458629		朋友	
☐	华兹因	16458795364	微信: 4wr865g15			

搜索:

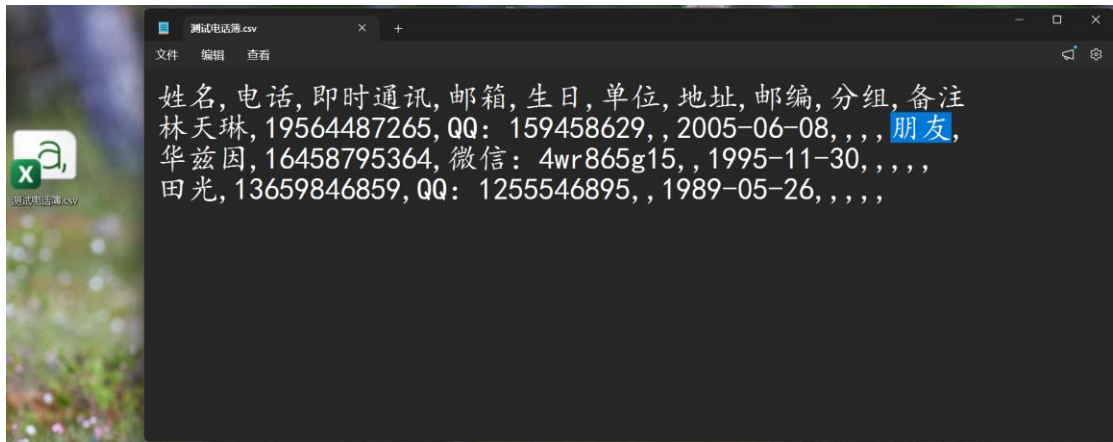
搜索重置

亦可对电话号码进行模糊搜索

5.5 导入导出



① 导出操作（可自由选择存入路径）



② 导出结果，并可以继续修改



③ 导入操作



④ 导入结果，并保存②的修改

5.6 批量操作



可通过勾选、拖动、或是“全选”按钮进行批量删除联系人、加入或移除分组

六、总结

李春良：

在本次《面向对象程序设计实践》课程的通讯录管理程序开发项目中，我参与了系统从需求分析、架构设计到功能实现与测试优化的完整流程，主要负责系统界面设计与实现工作。基于 Java Swing 技术栈，我完成了系统整体界面布局规划，搭建了包含顶部功能栏、左侧分组树、中间联系人表格与底部搜索栏的主窗口界面，实现了联系人编辑弹窗、分组树形菜单、数据展示表格等组件，通过组件排版、功能分区和统一的视觉风格，让系统操作更符合用户使用习惯。同时，我对界面交互进行了优化，采用功能配色区分按钮、统一提示弹窗样式，配合事件监听机制实现界面与业务逻辑的无缝对接，确保新增、修改、删除、搜索、导入导出等操作响应及时、友好反馈，提升了系统的简单性与稳定性。在开发过程中遵循 MVC 分层设计思想，将视图层与控制层、业务逻辑层、数据层解耦，通过模块化设计提高代码复用性与可维护性，也深刻理解了面向对象编程中封装、复用、扩展的核心思想。此次实践不仅让我学到了 Java Swing 界面开发、事件处理、异常交互等实用技能，更提升了需求分析、系统设计与团队协作能力，充分体会到理论知识与工程实践结合的重要性。今后我会继续优化界面设计能力，在美观与交互细节上不断完善，为开发更优质的程序打下基础。

黄奕恒：

在本次通讯录管理系统开发中，我主要负责后端业务逻辑与数据管理部分的实现。开发过程中，我围绕联系人管理、分组管理、数据持久化以及搜索筛选等核心功能进行了设计与编码，完成了 ContactService、GroupService、Repository 等模块的开发，实现了联系人新增、修改、删除、批量操作以及分组筛选等功能。同时，我通过 Java 序列化机制实现了本地数

据持久化，使系统能够自动保存并恢复联系人数据，并利用 CSV 工具类完成了联系人数据的导入导出功能。

在开发过程中，我进一步熟悉了 MVC 分层架构思想，理解了业务逻辑层、数据访问层之间的协作关系，也掌握了集合框架、文件 IO、异常处理等知识的实际应用。为了提高程序稳定性，我还参与了异常处理与数据校验功能的优化，减少程序运行中的错误情况。

不过，在项目测试阶段也发现了一些不足，例如导出的 CSV 文件在部分 Excel 环境下打开时会出现中文乱码问题，说明系统在文件编码兼容性方面还有待进一步优化。后续我希望通过 BOM 头处理或直接生成 Excel 文件等方式解决该问题，进一步提升系统的实用性与兼容性。